

1. Factorial

El factorial de un número n (denotado como $n!$) se define como el producto de todos los números enteros positivos desde 1 hasta n , o sea:

$$n! = \prod_{k=1}^n k$$

o lo que es lo mismo:

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \cdot (n-1)! & \text{si } n > 0 \end{cases}$$

Implemente un programa que reciba un número entero no negativo n de la consola y calcule el factorial de ese número.

Ejemplos

- Entrada: 0
Salida: El factorial de 0 es 1.
- Entrada: 1
Salida: El factorial de 1 es 1.
- Entrada: 5
Salida: El factorial de 5 es 120.
- Entrada: 7
Salida: El factorial de 7 es 5040.

2. Cantidad de dígitos

Implemente un método que reciba un número entero y halle su cantidad de dígitos (no usar la clase 'string').

3. Suma de Dígitos

Implementa un método que calcule recursivamente la suma de los dígitos de un número natural n . Si n tiene d dígitos, representados por $d_1, d_2, \dots, d_d$, entonces la suma de los dígitos $S(n)$ se define como:

$$S(n) = \sum_{i=1}^d d_i$$

Donde d_i representa el i -ésimo dígito de n .

4. Potencia

Implemente un método que dado dos enteros a y b , calcule el resultado de a^b .

5. Máximo común divisor

Implemente un método que halle el *máximo común divisor* entre dos enteros utilizando el algoritmo de Euclides basado en el resto.

El $\text{mcd}(a, b)$ es el mayor número d que divide a a y a b simultáneamente. Si $a = b \cdot q + r$, donde:

$$q \in \mathbb{Z}, \quad (\text{el cociente debe ser un número entero})$$

$$0 \leq r < b, \quad (\text{el residuo } r \text{ es un número no negativo y estrictamente menor que } b).$$

entonces:

$$\text{mcd}(a, b) = \text{mcd}(b, r).$$

Esto se debe a que cualquier divisor común de a y b también divide r , y viceversa.

6. Sucesión de Fibonacci

Implementa un método que halles el n -ésimo término de la sucesión de Fibonacci:

$$F_0 = 1$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \text{ para } n > 1$$

7. Secuencia de Collatz

Escriba un programa que lea un número entero n de la consola e imprima la secuencia de Collatz para n .

La secuencia de Collatz se define como:

$$S_0 = n$$
$$S_{n+1} = \begin{cases} 3S_n + 1 & \text{si } S_n \text{ impar} \\ \frac{S_n}{2} & \text{si } S_n \text{ par} \end{cases}$$

Dicha secuencia termina cuando se alcanza el número 1.

8. Superprimo

Implemente un método que determine si un número es super primo. Un número es super primo si es primo y además al quitarle la última cifra sigue siendo super primo. Un número primo de una sola cifra se considera super primo. Ejemplo, 73 es superprimo.

9. Representación binaria

a) Implemente un método que convierta un número de binario a decimal.

Un número binario es una representación en base 2 de un número entero. El número binario está compuesto solo por los dígitos 0 y 1, donde cada posición en el número tiene un valor que es una potencia de 2, comenzando desde la derecha (posición 0).

Para convertir un número binario a decimal, se puede utilizar la siguiente fórmula:

$$n = \sum_{i=0}^k b_i \cdot 2^i$$

donde b_i es el i -ésimo dígito del número binario, comenzando desde la derecha, y k es el índice de la posición más significativa (la izquierda).

Ejemplos

- Entrada: 1101
Salida: 13
Explicación:

$$1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = 13.$$

El número binario 1101_2 equivale a 13 en decimal.

- Entrada: 10101
Salida: 21
Explicación:

$$10101_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 16 + 0 + 4 + 0 + 1 = 21.$$

El número binario 10101_2 equivale a 21 en decimal.

- b) Implemente un método que reciba un número entero no negativo y devuelva un string con su representación binaria.

El número binario correspondiente se obtiene dividiendo el número entre 2 repetidamente, y registrando los restos de cada división. El número binario es el conjunto de los restos en orden inverso.

Ejemplos

- Entrada: 13
Salida: 1101
Explicación:

$$13 \div 2 = 6, \quad \text{resto } 1$$

$$6 \div 2 = 3, \quad \text{resto } 0$$

$$3 \div 2 = 1, \quad \text{resto } 1$$

$$1 \div 2 = 0, \quad \text{resto } 1$$

Los restos en orden inverso son 1101, por lo tanto, la representación binaria de 13 es 1101.

- Entrada: 21
Salida: 10101
Explicación:

$$21 \div 2 = 10, \quad \text{resto } 1$$

$$10 \div 2 = 5, \quad \text{resto } 0$$

$$5 \div 2 = 2, \quad \text{resto } 1$$

$$2 \div 2 = 1, \quad \text{resto } 0$$

$$1 \div 2 = 0, \quad \text{resto } 1$$

Los restos en orden inverso son 10101, por lo tanto, la representación binaria de 21 es 10101.

10. Echo invertido

Implemente un método que permita al usuario escribir en la consola tantas líneas como quiera. Cuando el usuario de "ENTER" sin haber escrito algún texto, debe imprimir cada una de las líneas que escribió el usuario, pero en orden inverso. No debe usar array ni ninguna estructura de datos para almacenar todas las líneas de texto que ha ido escribiendo el usuario.

11. Torres de Hanoi

¿Recuerdas el problema de las Torres de Hanoi? Este consiste en mover una pila de discos de una torre a otra, respetando las siguientes reglas:

- Solo se puede mover un disco a la vez.
- Un disco nunca puede colocarse sobre otro más pequeño.
- Solo se puede usar una torre auxiliar.

Escribe un método recursivo para resolver el problema de las Torres de Hanoi, mostrando los movimientos de los discos.

Ejemplo:

Si tienes 3 discos, la secuencia de movimientos sería:

- Mover disco 1 de A a C
- Mover disco 2 de A a B
- Mover disco 1 de C a B
- Mover disco 3 de A a C
- Mover disco 1 de B a A
- Mover disco 2 de B a C
- Mover disco 1 de A a C

12. Paréntesis balanceados

Implemente un método que, dado un entero n , calcule la cantidad de cadenas (de paréntesis abiertos y cerrados) balanceadas de longitud $2 \cdot n$.

Una cadena de paréntesis está balanceada si:

- Cualquier subcadena que comienza en el principio tiene un número de paréntesis abiertos mayor o igual que el número de paréntesis cerrados.
- La cadena tiene el mismo número de paréntesis abiertos que cerrados.

$((()))$ es una cadena balanceada y $()()$ no lo es.

Ejemplos

Para $n = 3$, hay 5 cadenas:

- $((()))$
- $(())()$
- $(())()$
- $()(())$
- $()()()$